Instruction Code

Instruction Code

- An instruction code is a binary code that tells the computer what operation to perform and where the data is located.
- Each instruction is stored in memory, and the processor reads (fetches) the instruction and executes it accordingly.

Parts of an Instruction:

An instruction consists of two parts:

- **Opcode (Operation Code):**
  Specifies the operation to be performed.
- **Address Field:**
  Specifies the memory location of the operand (data).

Example:

Instruction: ADD 457

Meaning:

Add the data stored at memory location 457 to the accumulator.

Operation:

AC ← AC + M[457]

Stored Program Concept

Stored Program Concept

- The stored program concept means that both instructions and data are stored in the same memory of the computer.
- In this concept, a program is a sequence of instructions stored in memory.
- The processor fetches the instructions one by one and executes them automatically.

Stored Program Organization:

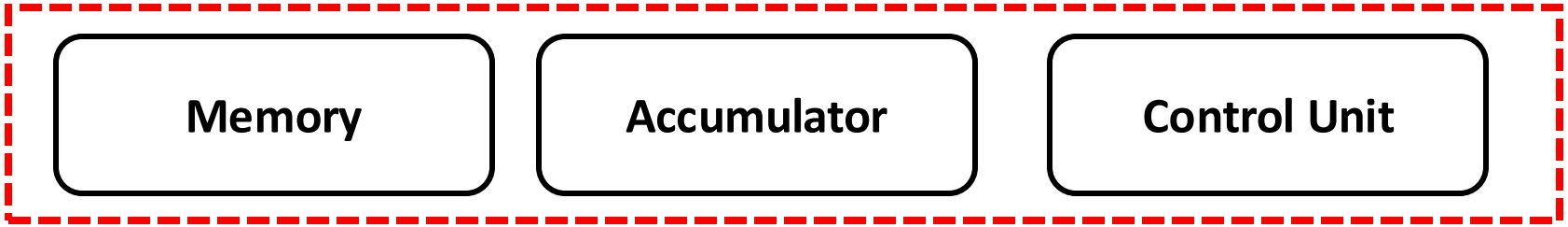


Fig. Stored Program Organization

- **Memory:** Stores both instructions and data (operands).
- **Accumulator (AC):** A processor register used for arithmetic and logic operations.
- **Control Unit (CU):** Interprets instructions and controls the operations of the computer.

ADITYA UNIVERSITY

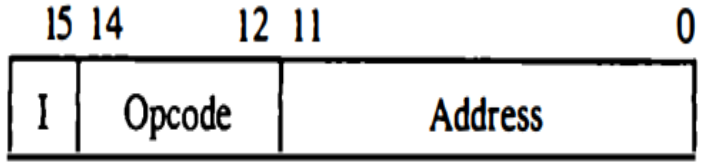
Computer Registers

- The registers are also listed in the Table together with a brief description of their function, the number of bits that they contain, and all these **registers are present in the CPU with a memory size of 4096*16.**

List of Registers for the Basic Computer

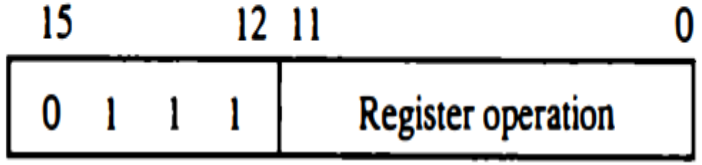
Register symbol	Number of bits	Register name	Function
DR	16	Data register	Holds memory operand
AR	12	Address register	Holds address for memory
AC	16	Accumulator	Processor register
IR	16	Instruction register	Holds instruction code
PC	12	Program counter	Holds address of instruction
TR	16	Temporary register	Holds temporary data
INPR	8	Input register	Holds input character
OUTR	8	Output register	Holds output character

Computer Instructions



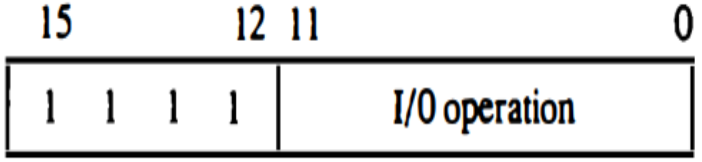
(Opcode = 000 through 110)

(a) Memory – reference instruction



(Opcode = 111, $I = 0$)

(b) Register – reference instruction



(Opcode = 111, $I = 1$)

(c) Input – output instruction

Symbol	Hexadecimal code $I = 0$	Hexadecimal code $I = 1$	Description
AND	0xxx	8xxx	AND memory word to *AC*
ADD	1xxx	9xxx	Add memory word to *AC*
LDA	2xxx	Axxx	Load memory word to *AC*
STA	3xxx	Bxxx	Store content of *AC* in memory
BUN	4xxx	Cxxx	Branch unconditionally
BSA	5xxx	Dxxx	Branch and save return address
ISZ	6xxx	Exxx	Increment and skip if zero
CLA	7800		Clear *AC*
CLE	7400		Clear *E*
CMA	7200		Complement *AC*
CME	7100		Complement *E*
CIR	7080		Circulate right *AC* and *E*
CIL	7040		Circulate left *AC* and *E*
INC	7020		Increment *AC*
SPA	7010		Skip next instruction if *AC* positive
SNA	7008		Skip next instruction if *AC* negative
SZA	7004		Skip next instruction if *AC* zero
SZE	7002		Skip next instruction if *E* is 0
HLT	7001		Halt computer
INP	F800		Input character to *AC*
OUT	F400		Output character from *AC*
SKI	F200		Skip on input flag
SKO	F100		Skip on output flag
ION	F080		Interrupt on
IOF	F040		Interrupt off

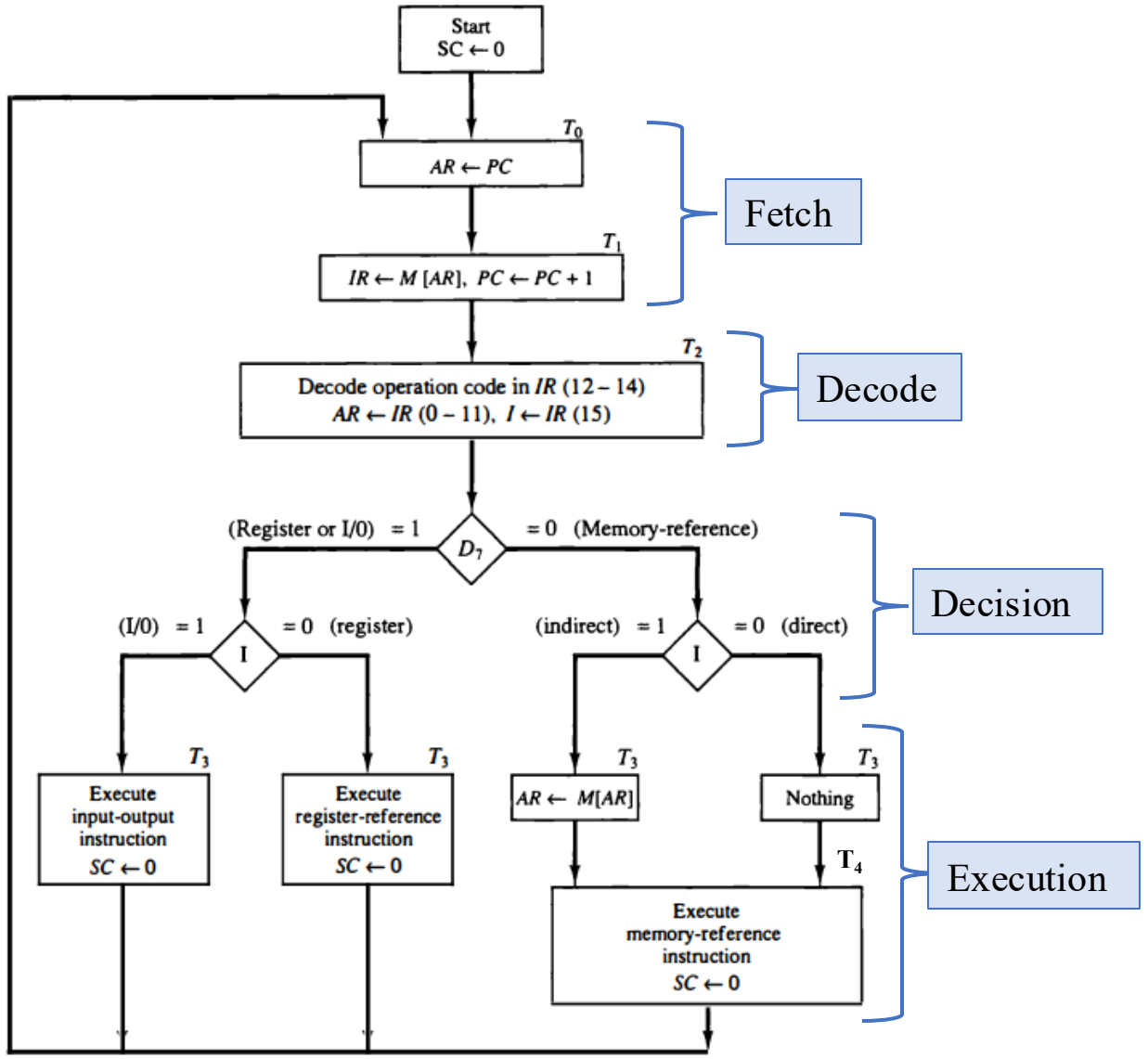
MRI

RRI

IOI

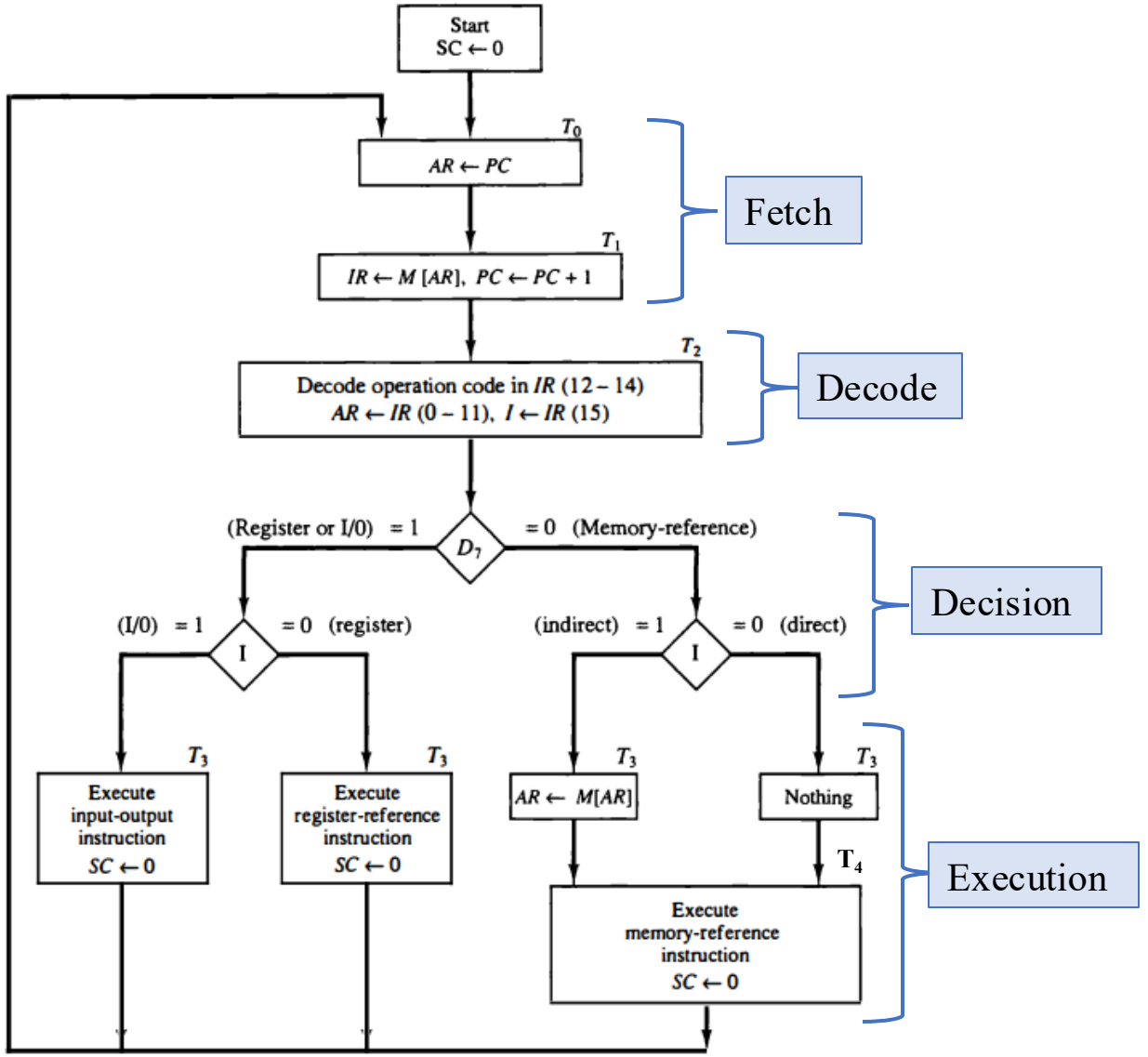
ADITYA UNIVERSITY

Instruction Cycle



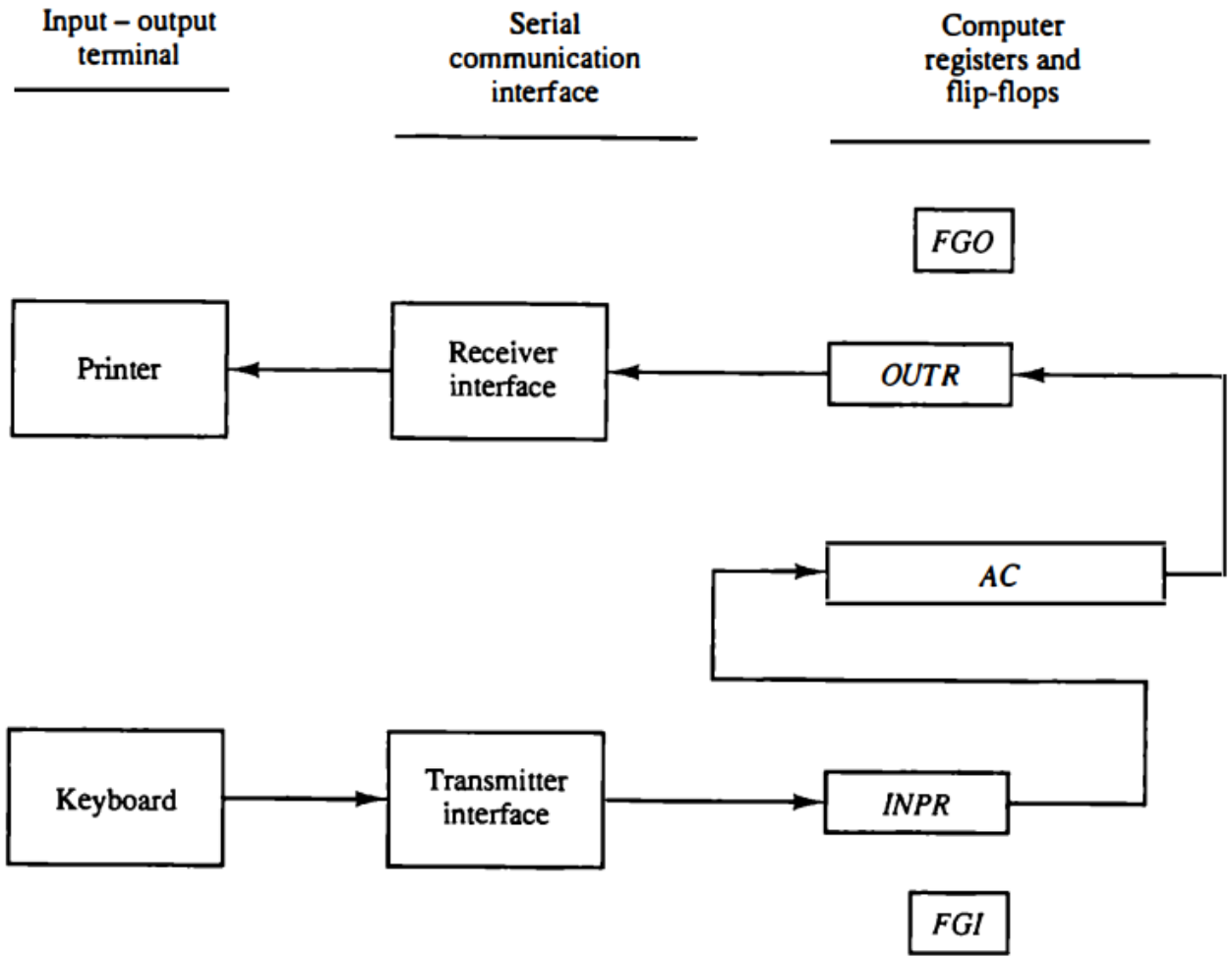
ADITYA UNIVERSITY

Instruction Cycle continue



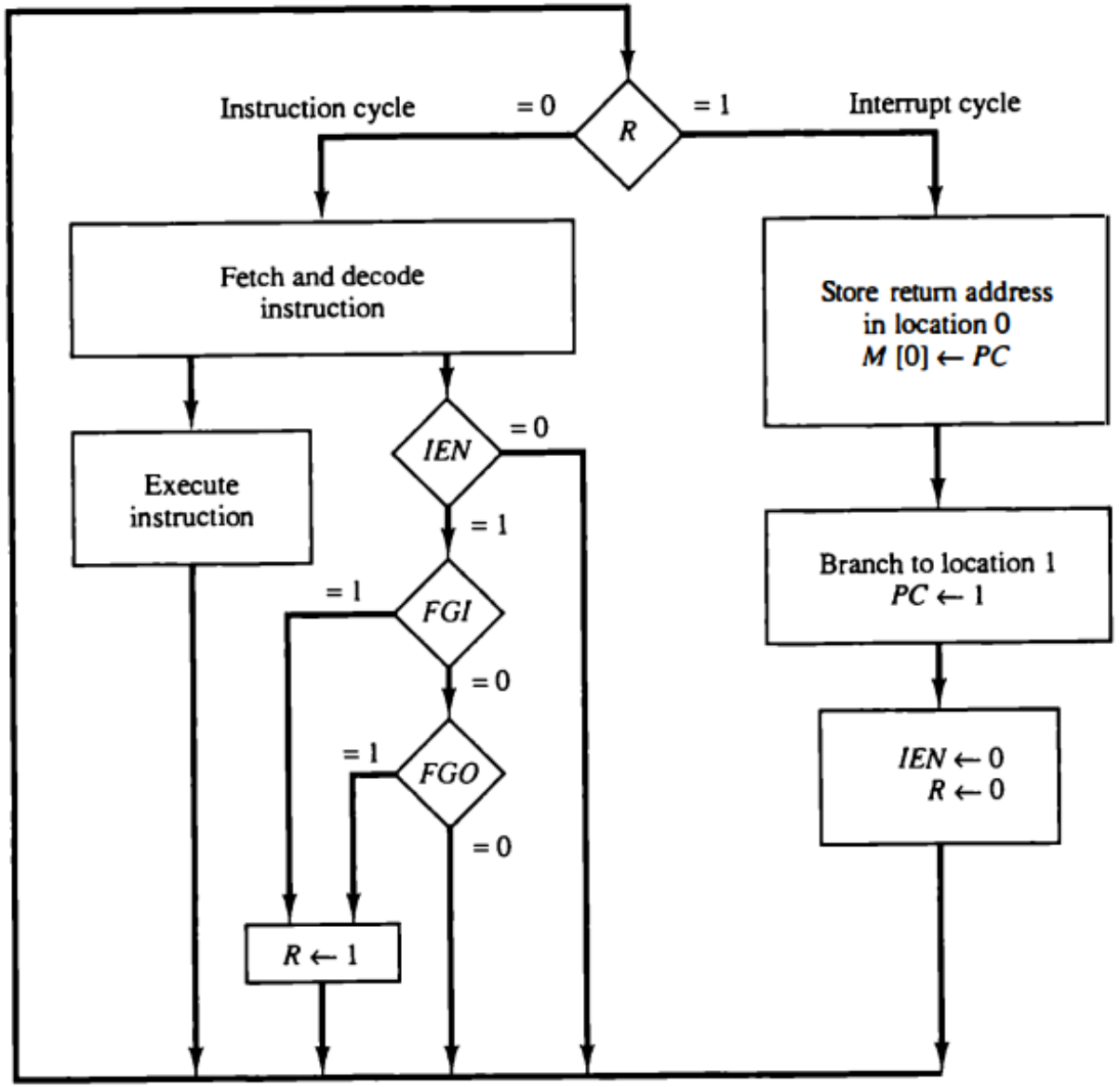
ADITYA UNIVERSITY

Input-Output Configuration



ADITYA UNIVERSITY

Input-Output and Interrupt

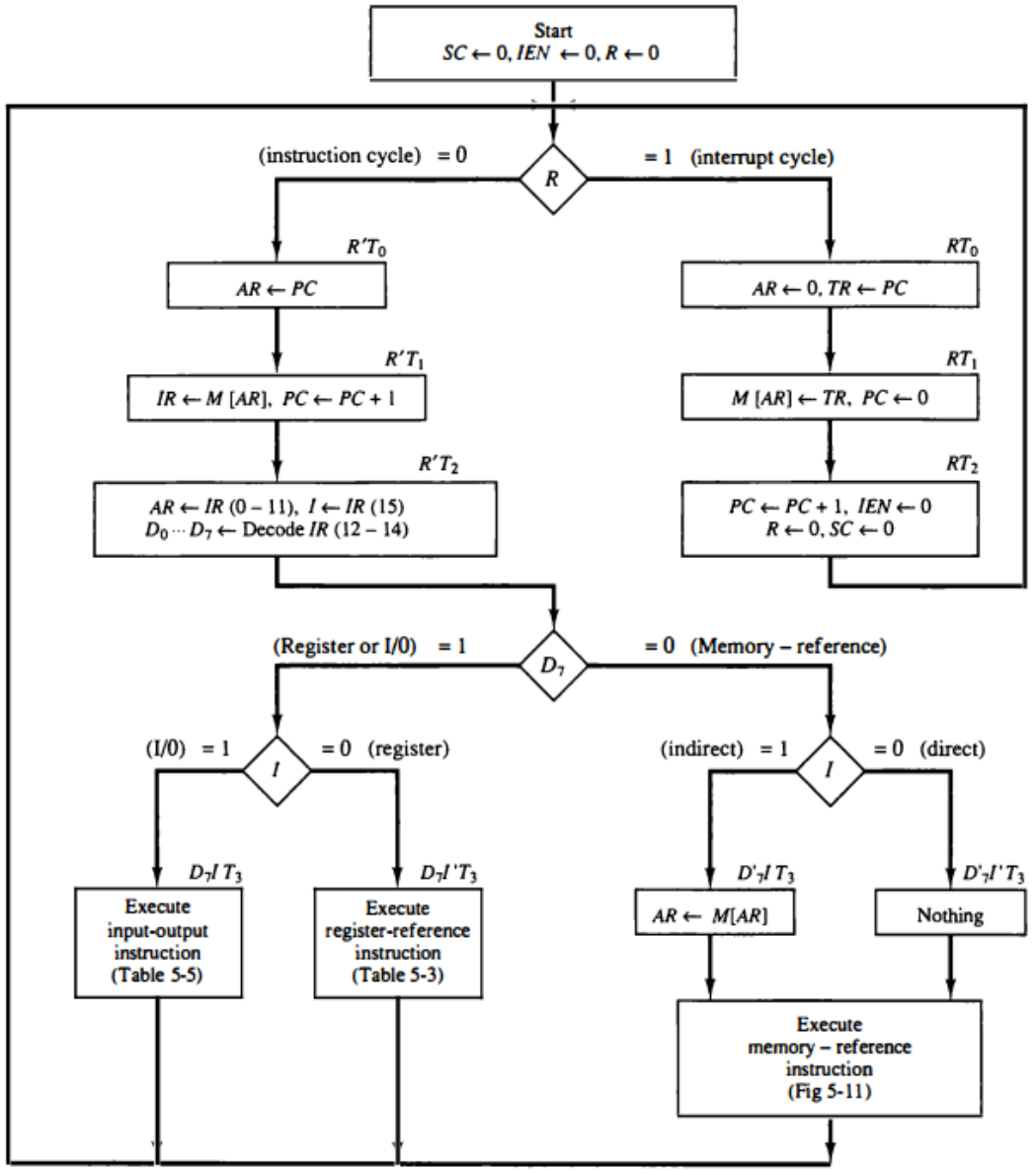


Complete Computer Description

- Complete Computer description, that is Instruction cycle including interrupt cycle.
- Instruction Cycle executes program → Interrupt Cycle handles urgent I/O → then CPU resumes program.
- Normal work = Instruction Cycle
  Urgent work (I/O) = Interrupt Cycle
  CPU switches automatically between both

Overall Flow:

- Fetch → Decode → Execute
- Check interrupt
- If interrupt:
  - Save PC
  - Jump to ISR
  - Handle I/O
- Return to main program



Complete Computer Description

1. Initial State

- SC ← 0 (Sequence Counter reset)
- IEN ← 0 (Interrupt disabled)
- R ← 0 (No interrupt request)

2. Instruction Cycle (When R = 0)

Step 1: Fetch

T_0: AR ← PC

T_1: IR ← M[AR], PC ← PC + 1

Instruction is fetched from memory

Step 2: Decode

T_2:

AR ← IR(0–11) (Address part)

I ← IR(15) (Mode bit)

D_0–D_7 ← Decode IR(12–14)

Instruction type is identified

Step 3: Execute

Based on instruction type:

(a) Memory Reference (D_7 = 0)

- If indirect: AR ← M[AR]
- Execute memory instruction

(b) Register Reference (D_7 = 1, I = 0)

- Perform register operation

(c) I/O Instruction (D_7 = 1, I = 1)

- Perform input/output operation

3. Interrupt Check

After execution:

Check **IEN = 1**

Check **FGI or FGO = 1**

If both conditions satisfied →

R ← 1 (Interrupt request generated)

4. Interrupt Cycle (When R = 1)

Step 1: Save Return Address

- **T_0:** AR ← 0, TR ← PC
- **T_1:** M[AR] ← TR, PC ← 0

Current program address is saved

Step 2: Branch to ISR

T_2:

PC ← PC + 1 (PC becomes 1 → ISR start)

IEN ← 0 (Disable interrupts)

R ← 0, SC ← 0

CPU starts interrupt service routine